

Loja Olinda Leila 2.0

O inimigo agora é outro

WHEN YOU ARE A PARENT



BUT ECOMMERCE IS LIFE

Lembrando de little coisas...

- ▲ Nosso cliente é a **Dona Leila!**
Amiga de longa data de um dos pais membro do grupo
- ▲ Por que ela precisava do site?
 - ❑ **DIVULGAÇÃO DO SEU TRABALHO**
(UNIFICAR VISUALIZAÇÃO DOS PRODUTOS)
 - ❑ **MUITA VENDA POR FIADO...**
PROBLEMA DA INFORMALIDADE!



O que foi implementado?

▲ Checkout Automatizado (API Mercado Pago)

▲ Segurança & Autenticação (Tokens JWT)

▲ Armazenamento em Nuvem (Integração Cloudinary)

▲ Serviços de email.

▲ Fluxo de Compras & Controle de Estoque

▲ Modelagem de Dados Relacional (PostgreSQL + Sequelize)

Automação Logística (WhatsApp + Visão Computacional)

TRADUÇÃO:

FRONT: Boniteza do site

Life IS sunshine and
rainbows



BACK: TODO O **RESTO!!!!!!**



Ideia princípio e TUDO!

▲ OPERAÇÕES BÁSICAS SOBRE... **MODELOS!**

- LER (READ)
- CRIAR
- ATUALIZAR (WRITE)
- DELETAR

▲ UPLOAD E coisas em NUVEM

- O que usamos para
fazer um e-commerce
FUNCIONAL de
GRAÇA



☐ Dev Backend Lore

Hierarquia de arquivos

- ❏ controller
 - ❏ `auth.controller.js`
 - ❏
- ❏ routes
 - ❏ `auth.routes.js`
 - ❏
- ❏ models
 - ❏ `auth.model.js`
 - ❏
- ❏ middlewares
 - ❏ `auth.middleware.js`
- ❏ config
 - ❏ `cloudinary.js`
- ❏ service
 - ❏ `email`



Motivo principal de termos uma hierarquia “de arquivos” é por conta da **organização**. ~DUH



Nossa hierarquia

- ❑ Cloudinary é para o upload e storage em cloud das imagens.
- ❑ Rotas definem as “URL”s que iremos testar no POSTMAN mais tarde
- ❑ Email service é chamado para todo o caso que queremos usar a API do Brevo para enviar um email para o cliente.
- ❑ server.js está embaixo. Define todo o backend. Chama os routers.

Requisitos mínimos

▲ CREATE

▲ READ

▲ UPDATE

▲ DELETE

▲ UPLOAD
DE
IMAGENS

☐ E PARA REALIZAR A IMPLEMENTAÇÃO DO CRUD, PRECISAMOS ANALISAR NOSSOS **MODELOS**!

☐ E SE TRATANDO DE UM E-COMMERCE O PADRÃO DOS MODELOS SERIA:

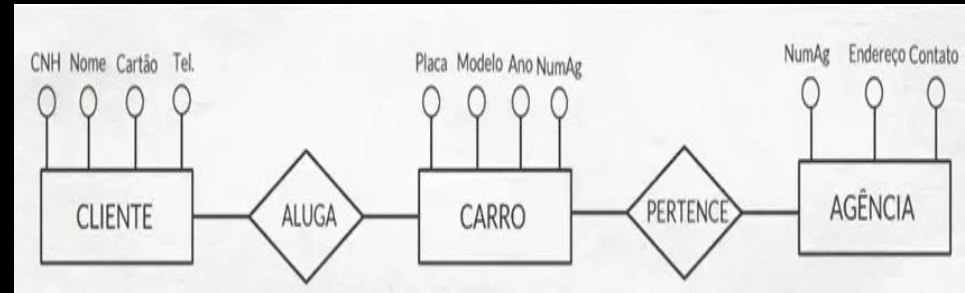
tá no próximo slide



Modelos

▲ Atributos

- ❑ Cria-se o conceito de “modelo”, pois **objetos** do site dispõem de atributos em comum, como:



▲ **Clothing**
(roupa),
nosso
produto:

Toda roupa a ser vendida tem:

- identificador único (id)
- nome
- tamanho (size)
- preço (price)
- **IMAGEM**
- estoque

- **COMO** UPAR IMAGEM?
- **ONDE** UPAR?
- **QUEM** TEM AUTORIZAÇÃO PARA UPAR?

Modelos no Sistema

MODELOS

- ❑ Existem no sistema INTEIRO para “sempre”.
- ❑ COMUM a todos os usuários.
- ❑ ENTIDADES!

DEFINIÇÕES

- ❑ Existem para as entidades!
- ❑ São como constantes de ALGUÉM.
- ❑ É um “modelo



Modelos

▲ Roupas

- ❑ ID
- ❑ Nome
- ❑ Preço
- ❑ Tamanho
- ❑ Imagem
- ❑ Quantidade em estoque

▲ Cart

- ❑ ID
- ❑ quantidade
- ❑ ID do **usuário**
- ❑ ID da **roupa**

▲ Pedido

- ❑ ID
- ❑ Preço total
- ❑ Itens
- ❑ Status
- ❑ Horário de criação
- ❑ Horário da última atualização
- ❑ ID do **usuário**

▲ Usuário

- ❑ ID
- ❑ Nome de usuário
- ❑ Senha
- ❑ Horário de criação
- ❑ Horário da última atualização
- ❑ É admin?
- ❑ Código de redefinição de senha
- ❑ Horário limite de redefinição de senha

CRUD

▲ C - Create (Criar)

- ❑ A Leila adiciona novos produtos

▲ R - Read (Ler)

- ❑ Os produtos são mostrados para os clientes

▲ U - Update (Atualizar)

- ❑ A Leila atualiza o estoque ou coisas específicas dos produtos

▲ D - Delete (Excluir)

- ❑ A Leila retira produtos que não estão mais disponíveis

O que foi usado para **deploy**?

▲ **Render** 

▲ **Neon**  **NEON**

▲ **Cloudinary** 

▲ **MERCADO PAGO** para o PIX 

▲ **Brevo** 

- ❑ O melhor é que são todos de graça! Sem pedir cartão.



“google list tools for deploy” para um dev miserável de pobre be like:



M My Workspace



Projects

Search ^ K

+ New

Upgrade



Projects

Blueprints

Environment Groups

INTEGRATIONS

Observability

Webhooks

Notifications

NETWORKING

Private Links

Dedicated IPs New

WORKSPACE

Billing

Settings

Changelog

Invite a friend

Contact support

Render Status

Overview

Invite your team

+ New

Projects

My project

No active services

+ Create new project

Ungrouped Services

Active (2)

Suspended (0)

All (2)

Search services

SERVICE NAME 2	STATUS	RUNTIME	REGION	UPDATED ↓	
api.lojaleila	✓ Deployed	Docker	Oregon	2h	...
lojaleila	✓ Deployed	Static	Global	9h	...



Guilherme

Free



loja-leila-db



All OK

Ask AI



Upgrade



PROJECT

Dashboard

Branches

Integrations

Settings

BRANCH

production



Overview

Monitoring

SQL Editor

Tables

Backup & Restore

Data Masking

BETA

APP BACKEND

Data API

Auth

Refined

Collapse menu

Tables

production

neondb



Schema

public



Search...



CartItems

Clothings

orders

users



+ Add record



0 - 0 of 0



Id serial



quantity integer



createdAt timestamp wi...



updatedAt timestamp wi...

No rows
limit 50 offset 0



dhzmy2wv



Home



Assets



Image



Video



MediaFlows



Moderation

Backup

Optimization

Webhook Notifications

Security

Account settings

My Profile

Account

Role Management

Product Environments

User Management

Account Security

Billing

Plan Details

Billing Information

Explore

Add-ons

Plan Details

Any questions? [Get in touch](#) and let us know how we can help.

Need to cancel your account? [Start the process here](#)

Base Plan



Image and Video

Free

\$0

0.16 / 25 Credits ⓘ

0.64% monthly usage

Next Bill Date

-

Change Plan



MediaFlows

Free

\$0

Next Bill Date

-

Change Plan

Total

\$0



Integrações

Gerencie suas aplicações e credenciais de acesso.

[Suas aplicações](#)[Aplicações de outras contas](#)[+ Criar aplicação](#)

Aplicação

Qualidade da Integração ?

**LojaLeila**

Número da aplicação: 7187243296236049

Abaixo do ideal



Olá, Loja

🔧 Personalizar página

👉 Fazer upgrade agora

junho 2026						
<						>
seg	ter	qua	qui	sex	sáb	dom
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	1	2	3	4	5

Comece a usar a Brevo

Criar campanha



Adicione seus primeiros contatos

Você precisa de contatos para criar uma campanha. Crie seu banco de dados de contatos ou adicione os destinatários da sua primeira campanha.

✔ Feito

Seus contatos

Adicionar contato

2

Total de contatos



Uso de seu plano

E-mails



300 restantes de 300 até 08/06/2026

Créditos pré-pagos



0 créditos restantes

Status code: **OK**

▲ Quando dá return **OK** no /auth

- ☐ Login realizado com sucesso
- ☐ Logout realizado com sucesso
- ☐ getAllUsers foi um sucesso
- ☐ getUserProfile foi um sucesso
- ☐ Perfil atualizado com sucesso
- ☐ Conta do usuário removida com sucesso
- ☐ Senha atualizada com sucesso
- ☐ Email de recuperação de senha enviado com sucesso
- ☐ Senha redefinida com sucesso
- ☐ ...



200
OK

Status code: **CREATED**

▲ Quando retorna **CREATED**

- ❑ no **/orders** quando o **pedido** é criado com sucesso.
- ❑ no **/clothings** quando a roupa é cadastrada com sucesso.
- ❑ e no **/auth** quando o **usuário** é cadastrado com sucesso.

▲ Por que o **/cart** difere?

- ❑ O **addToCart** retorna **OK**, pois ele altera o estado de um recurso do usuário. Ele não cria uma entidade nova no sistema.



201
Created



Guilherme

Free



loja-leila-db



All OK

Ask AI



Upgrade



PROJECT

Dashboard

Branches

Integrations

Settings

BRANCH

production



Overview

Monitoring

SQL Editor

Tables

Backup & Restore

Data Masking

BETA

APP BACKEND

Data API

Auth

Collapse menu

Tables

production

neondb



Schema

public



Search...



CartItems

Clothings

orders

users



+ Add record



0 - 0 of 0



Id serial



quantity integer



createdAt timestamp wi...



updatedAt timestamp wi...

No rows
limit 50 offset 0

lojaRoupa

backend

node_modules

public/uploads

src

config

controllers

middlewares

models

CartItem.model.js

clothing.model.js

dbconfig.js

order.model.js

user.model.js

routes

services

.dockerignore

.env

.gitignore

Dockerfile

{ } package.json

{ } package-lock.json

server.js

frontend

{ } package.json

{ } package-lock.json

README.md

← →

hing.model.js

auth.routes.js

auth.controller.js

email.service.js

order.routes.js

server.js

CartItem.model.js

+

□

↗

backend/src/models/CartItem.model.js > const CartItem > id

Q

I

⚙

```
1 import { DataTypes } from "sequelize";
2 import sequelize from "../dbconfig.js";
3 import User from "../user.model.js";
4 import Clothing from "../clothing.model.js";
5
6 const CartItem = sequelize.define("CartItem", {
7   id: {
8     type: DataTypes.INTEGER,
9     primaryKey: true,
10    autoIncrement: true,
11  },
12  quantity: {
13    type: DataTypes.INTEGER,
14    allowNull: false,
15    defaultValue: 1,
16    validate: {
17      min: 1,
18    },
19  },
20 });
21
22 // Relacionamentos
23 User.hasMany(CartItem, { foreignKey: "userId", onDelete: "CASCADE" });
24 CartItem.belongsTo(User, { foreignKey: "userId" });
25
26 // ... (rest of the code is partially visible and cut off)
```



lojaRoupa

backend

node_modules

public/uploads

src

config

controllers

middlewares

models

CartItem.model.js

clothing.model.js

dbconfig.js

order.model.js

user.model.js

routes

services

.dockerignore

.env

.gitignore

Dockerfile

{ } package.json

{ } package-lock.json

server.js

frontend

{ } package.json

{ } package-lock.json

README.md



clothing.model.js

auth.routes.js

auth.controller.js

email.service.js

order.routes.js

server.js

CartItem.model.js

.git

backend/src/models/clothing.model.js



```
1 import { Model, DataTypes } from "sequelize";
2
3 import sequelize from "../dbconfig.js";
4
5 class Clothing extends Model {}
6
7 Clothing.init(
8   {
9     id: {
10       type: DataTypes.INTEGER,
11       autoIncrement: true,
12       primaryKey: true,
13     },
14     name: {
15       type: DataTypes.STRING,
16       allowNull: false,
17     },
18
19     price: {
20       type: DataTypes.DECIMAL(10, 2),
21       allowNull: false,
22     },
23
24     size: {
25       type: DataTypes.ENUM("P", "M", "G"),
26       allowNull: false,
```


Status code: **Unauthorized**

▲ Quando retorna **Unauthorized**?

- ❑ Quando tentamos realizar uma ação no sistema que depende de confirmarmos quem somos no sistema. Se quem falamos que somos, realmente somos, para realizar a ação por aquele, ou para aquele que somos! SOMOS!
- ❑ Utilizador **não autenticado?** **Não possui Token?** **RETORNA 401** também!



401

Unauthorized

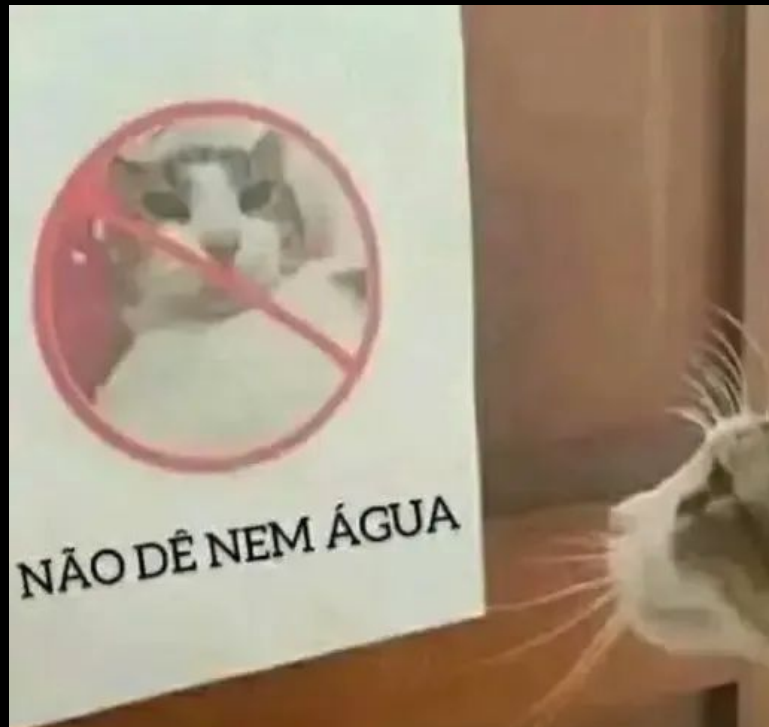
Autorização != Autenticação

▲ Autorização é feita pelo “Token”?

- ❑ O “Token” é pego do localStorage do usuário e é usado para validar se o usuário está autenticado. Logo se está logado.
- ❑ Serve para não ficar passando o { email, senha } para o servidor toda hora. **REST API é stateless.**

▲ MIDDLEWARE!

- ❑ Usa-se um middleware de **VerifyAdmin** e **VerifyToken**, ambos do **/auth**



Autenticação é pré-requisito

▲ /login

- ❑ Deve retornar o “Token”, logo o backend deve processar email e password válidos (encontrados no BD)

▲ /register

- ❑ Se o nome e o email JÁ NÃO estão registrados, registre.

▲ Falar do **Limitter** e de Injection code.
BENDITA ORM!!!



shutterstock.com • 764840686



- ❑ “Segurança da OLX” numa fila.



- ❑ “Segurança da OLX” deve olhar desconfiado para quem quer entrar.

Status code: **FORBIDDEN**

▲ Quando retorna **FORBIDDEN**?

- ❑ No `/auth` temos que ter uma verificação antes de acessarmos o servidor, algo que fique no meio. **MIDDLEWARE**.
- ❑ `auth.middleware.js` verifica se o usuário tem permissão suficiente para realizar determinada ação, como:
 - ❑ Criar roupa
 - ❑ Upload de imagem
 - ❑ Atualizar o status do pedido do cliente (lembrar de como a lojinha funciona)
 - ❑ Deletar roupa
 - ❑ Mostrar todos os usuários
 - ❑ ...



403
Forbidden

Status code: **NOT FOUND**

▲ Quando retorna **NOT FOUND**?

Erro de **BAD REQUEST**.

- ❑ No **/auth**
 - ❑ **GET**: Erro ao carregar perfil do usuário
 - ❑ **PUT**: Erro ao carregar perfil do usuário (update-profile)
 - ❑ **DEL**: Usuário não encontrado
 - ❑ **PUT**: Usuário não encontrado (change-password)
- ❑ No **/cart**
 - ❑ **DEL**: Item não encontrado no carrinho



404
Not Found

Status code: Internal Server Error

▲ Quando retorna **Internal Server Error**?

- ❑ Quando a culpa é do servidor, provavelmente.
- ❑ no `/auth` por exemplo quando tentamos encontrar todos os usuários e retornar para o admin: **`getAllUsers`**, se não der certo essa tentativa **`TRY`**, retornamos erro.

Tem diversos exemplos disso. Toda funcionalidade do **servidor** deve esperar por um **erro** do **próprio**.



500

Internal Server Error

Status code: Gateway Timeout

Quando retorna **Gateway Timeout**?

- ❑ ESPERO QUE NUNCA! Mas já aconteceu quando usado uma API que não respondia. **API KEY estava incorreta**. O servidor que continha a API **bloqueia acessos de IP's quaisquer**, tinha que permitir entrada.



504

Gateway Timeout

```
15  const app = express();
16  app.set("trust proxy", 1);
17
18  app.use(cors({ origin: "https://lojaleila.onrender.com" }));
19  app.use(express.json()); // Apresentacao - formatacao padrao
20  app.use(express.urlencoded({ extended: true }));
21
```

- ❑ Parecido com o cors, não ENGUAL.

CRUD users: /api/auth

The screenshot displays the Postman application interface. On the left, a sidebar shows a list of collections under the heading 'COLLECTIONS', including 'CLOTHINGS', 'ORDERS', and 'USERS'. The 'USERS' collection is expanded, showing a list of requests: 'GET Get data' (selected), 'POST Login', 'POST Login Copy', and 'DEL New Request'. The main workspace is divided into two panes. The top pane shows the 'GET Get data' request setup. The URL is 'https://api-lojalella.onrender.com/api/auth/usuarios'. The 'Auth Type' is set to 'Bearer Token', and a token is entered in the 'Token' field: 'IveFj9ObxwsAt0NwYt-fPMEIxBjIU'. Below the request setup, the response body is displayed in JSON format, showing a list of two user objects. The bottom pane shows the response status '200 OK' and the response body in JSON format. On the right side, there is a 'New Chat' section with a text input field and a 'Send' button. Below this, there are three buttons: 'Fix failing Get data (500)', 'Complete Login auth verification', and 'Add targeted tests for API errors'. At the bottom right, there is a chat window with a text input field and a 'Send' button. The chat window contains the text: 'Describe what you need. Press @ for context, / for Skills.' and a button labeled 'Auto'.

Guilherme Amnesia's Workspace

Search

POST Login POST Login Copy GET Get data DEL New Request

USERS > Get data

GET https://api-lojalella.onrender.com/api/auth/usuarios

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Auth Type: Bearer Token

Token: IveFj9ObxwsAt0NwYt-fPMEIxBjIU

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body: 200 OK • 2.36 s • 568 B

JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "username": "testeAdmin",
5     "isAdmin": true,
6     "createdAt": "2026-06-05T02:01:14.847Z"
7   },
8   {
9     "id": 3,
10    "username": "AdrianaRana",
11    "isAdmin": false,
12    "createdAt": "2026-06-07T03:00:43.041Z"
13  }
14 ]
```

Fix failing Get data (500)

Complete Login auth verification

Add targeted tests for API errors

Describe what you need. Press @ for context, / for Skills.

Auto

CRUD users: /api/auth

The screenshot displays the Postman application interface. On the left, a sidebar lists various collections and environments. The main workspace is titled "Guilherme Amnesia's Workspace" and contains a REST client request for the endpoint `https://api-fojalella.onrender.com/api/auth/change-password`. The request is a `PUT` method with a raw JSON body. The response is a `200 OK` status with a JSON body containing a success message.

Request Details:

- Method: `PUT`
- URL: `https://api-fojalella.onrender.com/api/auth/change-password`
- Body Type: `raw`
- Body Content:

```
1 {
2   "senhaAtual": "login1337senha",
3   "novaSenha": "NovaSenhaSuperSegura123"
4 }
```

Response Details:

- Status: `200 OK`
- Time: `3.07 s`
- Size: `529 B`
- Body Type: `JSON`
- Body Content:

```
1 {
2   "message": "Senha atualizada com sucesso!"
3 }
```

CRUD **users**: /api/auth

The screenshot displays the Postman application interface. On the left, a sidebar shows a collection named 'USERS' with a sub-collection 'Login Copy' selected. The main workspace shows a POST request to 'https://api-lojalella.onrender.com/api/auth/register'. The request body is a JSON object with the following details:

```
1 {
2   "username": "AdrianaRana",
3   "email": "guilhermeonlyone@gmail.com",
4   "password": "login1337senha"
5 }
```

The response is a 201 Created status with a JSON body:

```
1 {
2   "message": "Usuário cadastrado com sucesso!"
3 }
```

At the bottom right, a chat window is visible with the text: 'Describe what you need. Press @ for context, / for Skills.' and an 'Auto' button.

CRUD users: /api/auth

The screenshot displays the Postman API client interface. On the left, a sidebar shows a collection named 'USERS' with four requests: 'GET Get data', 'POST Login', 'POST Login Copy', and 'GET New Request'. The 'GET New Request' is selected. The main panel shows a 'New Request' for the 'USERS' collection, with the method set to 'DELETE' and the URL 'https://api-lojalella.onrender.com/api/auth/usuarios/2'. The 'Authorization' tab is active, showing 'Bearer Token' as the auth type and a token 'f17EgwePZI-0UxyzTtToCpqaLSphM'. Below this, a message states: 'The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.' The 'Body' tab shows a JSON response:

```
{  "message": "Conta de usu\u00e1rio removida do sistema com sucesso."}
```

 The status bar indicates a '200 OK' response. On the right, a 'New Chat' panel is visible with a text input area and a 'Run a quick auth smoke test' button.

CRUD **users**: /api/auth

The screenshot displays the Postman interface with a workspace named "Guilherme Amnesia's Workspace". The left sidebar shows a collection of API endpoints under the "USERS" folder, including "Get data", "Login", "register", "New Request", "perfil", and "forgot". The "forgot" endpoint is selected, showing a POST request to "https://api-lojalella.onrender.com/api/auth/forgot-password". The request body is a JSON object:

```
{  "email": "guilhermeonlyone@gmail.com"}
```

. The response is a 200 OK status, indicating success, with a message in Portuguese:

```
{  "message": "Se o e-mail existir em nossa base, um link de recupera\u00e7\u00e3o ser\u00e1 enviado."}
```

. The bottom status bar shows "Connect Git", "Console", "Terminal", and "0 0 0" indicators.

Workspace: Guilherme Amnesia's Workspace

Collection: USERS > forgot

Method: POST URL: https://api-lojalella.onrender.com/api/auth/forgot-password

Body (raw):

```
1 {
2   "email": "guilhermeonlyone@gmail.com"
3 }
```

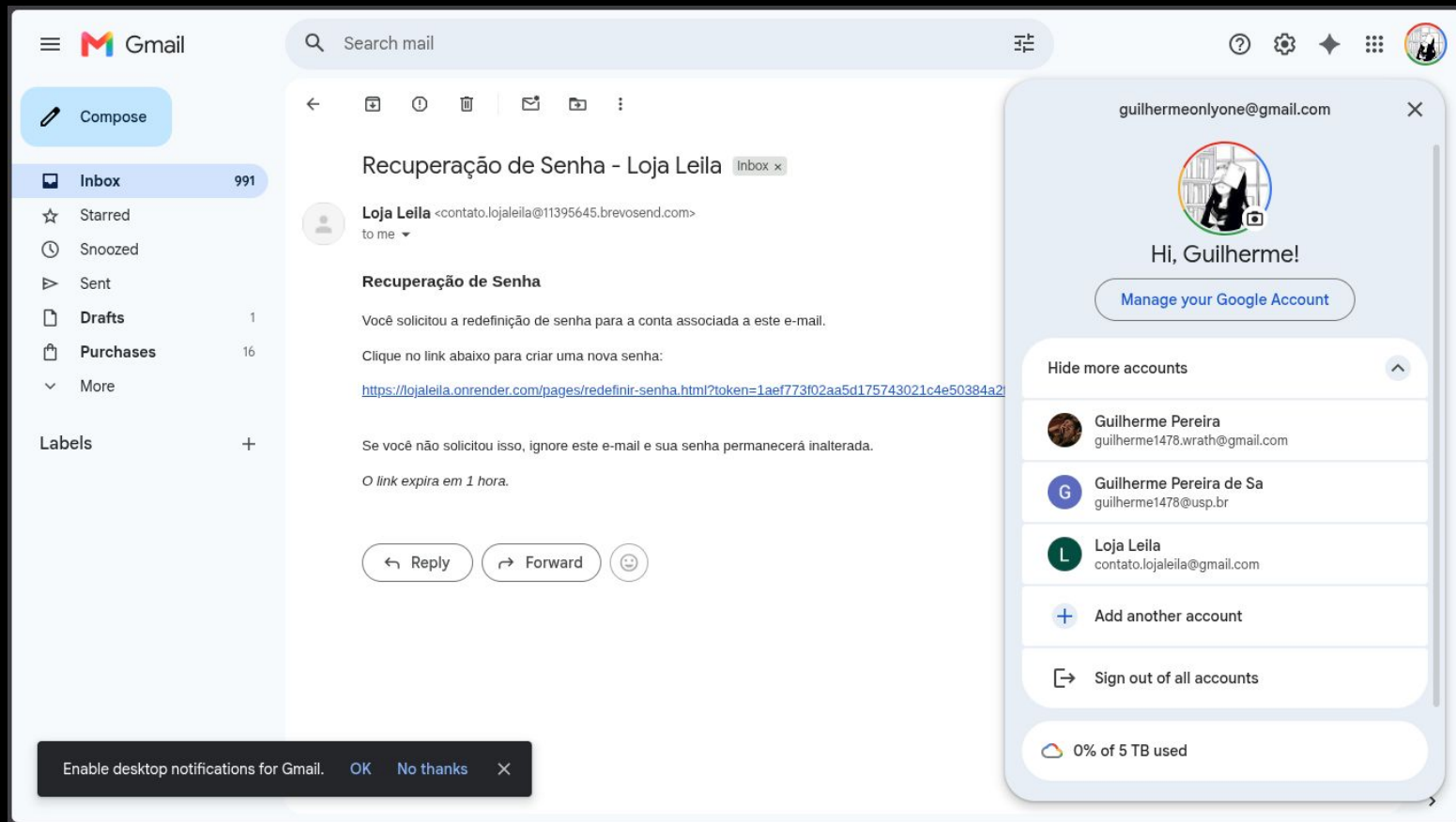
Response: 200 OK • 3.18 s • 564 B

Body (JSON):

```
1 {
2   "message": "Se o e-mail existir em nossa base, um link de recupera\u00e7\u00e3o ser\u00e1 enviado."
3 }
```

Bottom Bar: Connect Git | Console | Terminal | 0 0 0

CRUD users: /api/auth



CRUD **users**: /api/auth

The screenshot displays the Postman application interface. On the left, a sidebar shows a collection of requests under the 'USERS' folder, including 'GET Get data', 'POST Login', 'POST Login Copy', and 'DEL Delete data'. The main panel shows the 'GET Get data' request configured with the URL 'https://api-lojalella.onrender.com/api/auth/usuarios' and 'Bearer Token' authorization. The response is a 401 Unauthorized status with a JSON body:

```
{  "message": "Acesso negado. Token não fornecido."}
```

. On the right, a 'New Chat' panel is visible, containing a text input field and a 'Run a quick auth smoke test' button.

Guilherme Amnesia's Workspace

Search

POST Login POST Login Copy GET Get data

USERS > Get data

GET https://api-lojalella.onrender.com/api/auth/usuarios

Auth Type: Bearer Token

Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

401 Unauthorized • 219 ms • 510 B

JSON Preview

```
1 {
2   "message": "Acesso negado. Token não fornecido."
3 }
```

Describe what you need. Press @ for context, / for Skills.

Auto

CRUD **users**: /api/auth

The screenshot displays the Postman API client interface. The top navigation bar includes a search bar, an 'Invite' button, an 'Upgrade' button, and various utility icons. The left sidebar shows a collection of API collections, with 'USERS' expanded and 'GET Get data' selected. The main workspace is divided into several tabs: 'POST Login', 'POST Login Copy', and 'GET Get data'. The 'GET Get data' tab is active, showing a request to 'https://api-lojalella.onrender.com/api/auth/usuarios'. The request is configured with a 'GET' method and a 'Bearer Token' authorization type. The token value is '0giWRFNZksd0-EeK7VXJ5Lz3IAzA'. The response is a '403 Forbidden' status with a message in Portuguese: 'Acesso irrestrito. Apenas administradores podem realizar essa ação'. The bottom status bar shows 'Connect Git', 'Console', 'Terminal', and '0 0 0'.

Guilherme Amnesia's Workspace

Search

POST Login POST Login Copy GET Get data

USERS > Get data

GET https://api-lojalella.onrender.com/api/auth/usuarios

Auth Type: Bearer Token

Token: 0giWRFNZksd0-EeK7VXJ5Lz3IAzA

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

403 Forbidden • 1.64 s • 525 B

JSON Preview Try with different auth credentials

```
1 {
2   "message": "Acesso irrestrito. Apenas administradores podem realizar essa ação"
3 }
```

Run a quick auth smoke test

Add tests to the Order GET meus

Standardize request scripts across ORDERS

Describe what you need. Press @ for context, / for Skills.

Auto

Connect Git Console Terminal 0 0 0

Globals Vault Tools

CRUD **clothings**: /api/produtos

The screenshot displays the Postman application interface. On the left, a sidebar lists collections including 'Cart', 'CLOTHINGS', 'ORDERS', and 'USERS'. The 'CLOTHINGS' collection is expanded, showing a 'Get data' endpoint. The main panel shows the details of this endpoint, including the URL 'https://api-10jalella.onrender.com/api/produtos/1' and the 'Authorization' tab set to 'Bearer Token'. The response is a 200 OK status with a JSON body containing product details.

Request Details:

- Method: GET
- URL: https://api-10jalella.onrender.com/api/produtos/1
- Auth Type: Bearer Token
- Token: [Redacted]

Response Details:

- Status: 200 OK
- Time: 1.73 s
- Size: 625 B
- Body (JSON):

```
1 {
2   "id": 1,
3   "name": "Teste",
4   "price": "0.01",
5   "size": "M",
6   "image": "https://res.cloudinary.com/dhrzmy2wv/image/upload/v1786778386/lojaroupa_produtos/fpquoikro57alwm933yw.png",
7   "stock": 9
8 }
```

The bottom status bar shows 'Connect Git', 'Console', 'Terminal', and '0 0 0'.

CRUD **clothings**: /api/produtos

The screenshot displays a REST client interface with a sidebar on the left containing a collection of endpoints categorized by resource: Cart, CLOTHINGS, ORDERS, and USERS. The 'CLOTHINGS' category is expanded, showing endpoints for GET, POST, PUT, and DELETE. The main panel shows a PUT request to the endpoint `https://api-lojaleila.onrender.com/api/produtos/1`. The request body is a JSON object with the following structure:

```
1 {
2   "name": "Vestido Floral Leila Elegante",
3   "price": 1.01,
4   "size": "G",
5   "stock": 15
6 }
```

The response is shown in the bottom panel, indicating a successful update with a status of 200 OK. The response body is a JSON object:

```
1 {
2   "message": "Produto atualizado com sucesso!",
3   "clothing": {
4     "id": 1,
5     "name": "Vestido Floral Leila Elegante",
6     "price": 1.01,
7     "size": "G",
8     "image": "https://res.cloudinary.com/dhrzmy2wv/image/upload/v1780778306/lojaroupa_produtos/fpqhoikro57alwm933yw.png",
9     "stock": 9
10  }
11 }
```

The interface also includes a top navigation bar with search and environment settings, and a bottom status bar with connection and terminal information.

CRUD **clothings**: /api/produtos

The screenshot shows the Postman interface with a workspace named "Guilherme Amnesia's Workspace". The left sidebar displays a collection of collections, including "CLOTHINGS", which is expanded to show "POST Post data" selected. The main area shows a POST request to "api-10jalella.onrender.com/api/produtos/criar". The request body is set to "form-data" and contains the following data:

Key	Type	Value	Description	Bulk Edit
name	Text	Teste		
price	Text	00.01		
size	Text	M		
image	File	unnamed.png		
stock	Text	3		

The response is shown in the "Body" tab, indicating a "201 Created" status. The response body is a JSON object:

```
1 {
2   "message": "Roupa cadastrada com sucesso!",
3   "clothing": {
4     "id": 4,
5     "name": "Teste",
6     "price": "0.01",
7     "size": "M",
8     "stock": 3,
9     "image": "https://res.cloudinary.com/dhrzmy2wv/image/upload/v1780952072/lojaroupa_produtos/qvrhnn6iwqsjq24rfyrd.png"
10  }
11 }
```

The bottom status bar shows "Connect Git", "Console", "Terminal", and "Globals Vault Tools".

CRUD **clothings**: /api/produtos

The screenshot displays the Postman interface with the following details:

- Workspace:** Guilherme Amnesia's Workspace
- Left Sidebar:**
 - COLLECTIONS:** Cart, CLOTHINGS (expanded), ORDERS, USERS
 - CLOTHINGS:** GET Get data, POST Post data, PUT Update data, **DEL Delete data** (selected)
- Request Tab:** CLOTHINGS > Delete data
- URL:** `https://api-lojaleila.onrender.com/api/produtos/4`
- Method:** DELETE
- Body:** raw (selected)
- Response:** 200 OK, 2.10 s, 529 B
- JSON Body:**

```
1 {
2   "message": "Produto removido com sucesso!"
3 }
```
- Bottom Bar:** Connect Git, Console, Terminal, 0 errors, 0 warnings

CRUD **cart**: /api/cart

The screenshot displays a REST client interface within a workspace named "Guilherme Amnesia's Workspace". The left sidebar lists various collections, including "Cart", "CLOTHINGS", "ORDERS", and "USERS". The "Cart" collection is selected, and the "GET Get data" endpoint is highlighted.

The main panel shows the details of the "GET Get data" endpoint. The URL is set to "https://api-lojaleila.onrender.com/api/cart/". The request body is empty, and the response is displayed in the "Body" tab, showing a JSON object representing a cart item.

The response status is "200 OK" with a response time of "2.15 s" and a size of "728 B". The response body is a JSON object:

```
1 [
2   {
3     "id": 2,
4     "quantity": 1,
5     "createdAt": "2026-06-08T20:21:16.423Z",
6     "updatedAt": "2026-06-08T20:21:16.423Z",
7     "userId": 5,
8     "clothingId": 1,
9     "clothing": {
10      "id": 1,
11      "name": "Vestido Floral Leila Elegante",
12      "price": "1.01",
13      "size": "G",
14      "image": "https://res.cloudinary.com/dhrzmy2wv/image/upload/v1780778306/lojaroupa_produtos/fpqhoikro57alwm933yw.png",
15      "stock": 9
16    }
17  }
18 ]
```

The bottom status bar shows "Connect Git", "Console", "Terminal", and "Globals Vault Tools".

CRUD **cart**: /api/cart

The screenshot displays the Postman interface with a workspace named "Guilherme Amnesia's Workspace". The left sidebar shows a collection named "Cart" with several endpoints. The "POST Post data" endpoint is selected, and its details are shown in the main panel. The request is a POST to "https://api-lojaleila.onrender.com/api/cart/add" with a raw JSON body. The response is a 200 OK status with a JSON body indicating a successful addition to the cart.

Request Details:

- Method: POST
- URL: https://api-lojaleila.onrender.com/api/cart/add
- Body (raw):

```
1 {
2   "clothingId": 1,
3   "quantity": 1
4 }
```

Response Details:

- Status: 200 OK
- Time: 53.26 s
- Size: 630 B
- Body (JSON):

```
1 {
2   "message": "Produto adicionado ao carrinho!",
3   "cartItem": {
4     "id": 2,
5     "userId": 5,
6     "clothingId": 1,
7     "quantity": 1,
8     "updatedAt": "2026-06-08T20:21:16.423Z",
9     "createdAt": "2026-06-08T20:21:16.423Z"
10  }
11 }
```

CRUD **cart**: /api/cart

The screenshot displays a REST client interface within a workspace named "Guilherme Amnesia's Workspace". The left sidebar lists collections: "Cart", "CLOTHINGS", "ORDERS", and "USERS". The "Cart" collection is expanded, showing endpoints for GET, POST, PUT, and DELETE. The "PUT Update data" endpoint is selected.

The main panel shows the "PUT" method being used on the URL "https://api-lojaleila.onrender.com/api/cart/update/2". The request body is a JSON object:

```
1 {
2   | "quantity": 2
3 }
```

The response is a 200 OK status, indicating a successful update. The response body is a JSON object:

```
1 {
2   | "message": "Quantidade atualizada com sucesso!",
3   | "cartItem": {
4     |   "id": 2,
5     |   "quantity": 2,
6     |   "createdAt": "2026-06-08T20:21:16.423Z",
7     |   "updatedAt": "2026-06-08T20:40:28.831Z",
8     |   "userId": 5,
9     |   "clothingId": 1
10  | }
11 }
```

The interface includes a "Send" button for the request and a "Save Response" button for the response. The bottom status bar shows "Connect Git", "Console", "Terminal", and "Globals Vault Tools".

CRUD orders: /api/pedidos

The screenshot displays the Postman application interface. On the left sidebar, the 'COLLECTIONS' list includes 'CLOTHINGS', 'ORDERS', and 'USERS'. The 'ORDERS' collection is expanded, showing a request named 'testando pedido confirmado'. The main panel shows the details of this PATCH request to the URL 'https://api-lojalella.onrender.com/api/pedidos/1/confirmar'. The 'Authorization' tab is active, showing a 'Bearer Token' type. Below this, a message states: 'The authorization header will be automatically generated when you send the request. Learn more about Bearer Token authentication'. The 'Body' tab shows a JSON response with a status of '200 OK'. The JSON body is as follows:

```
1 {
2   "message": "Pagamento confirmado com sucesso!",
3   "order": {
4     "id": 1,
5     "totalPrice": "0.01",
6     "items": [
7       {
8         "id": 1,
9         "name": "Teste",
10        "price": 0.01
11      }
12    ],
13    "status": "Pago",
14    "createdAt": "2026-06-06T21:32:24.306Z",
15    "updatedAt": "2026-06-07T01:00:16.943Z",
16    "userId": 1
17  }
18 }
```

On the right side, there is a 'New Chat' section with a text input area containing the prompt: 'Describe what you need. Press @ for context, / for Skills.' Below the input is an 'Auto' button. At the bottom right, there are buttons for 'Add saved auth token reuse', 'Build a Orders test suite', and 'Create a Runner for an order flow'.

CRUD orders: /api/pedidos

Guilherme Amnesia's Workspace

Search

Invited Upgrade

ORDERS > meus

GET `https://api-lojalella.onrender.com/api/pedidos/meus` Send

Docs Params Authorization Headers Body Scripts Settings Cookies

none

This request does not have a body

Body 200 OK • 1.70 s • 650 B Save Response

JSON Preview Visualize

```
57 {
58   "status": "Pendente",
59   "createdAt": "2026-06-06T22:22:45.672Z",
60   "updatedAt": "2026-06-06T22:22:45.672Z",
61   "userId": 1
62 },
63 {
64   "id": 1,
65   "totalPrice": "0.01",
66   "items": [
67     {
68       "id": 1,
69       "name": "Teste",
70       "price": 0.01
71     }
72   ],
73   "status": "Pago",
74   "createdAt": "2026-06-06T21:32:24.306Z",
75   "updatedAt": "2026-06-07T01:00:16.943Z",
76 }
```

Run ORDERS requests via Runner

Add assertions to Get data

Fix auth and reuse across requests

Describe what you need. Press @ for context, / for Skills.

Auto

Connect Git Console Terminal 0 0

Globals Vault Tools

CRUD orders: /api/pedidos

The screenshot displays the Postman application interface. On the left sidebar, under the 'COLLECTIONS' section, the 'ORDERS' collection is expanded, showing several requests: 'GET Get data' (selected), 'POST Post data', 'PATCH testando pedido confirmado', 'GET meus', and 'PUT New Request'. The main workspace shows the details of the selected 'GET Get data' request. The URL is 'https://api-lojaleila.onrender.com/api/pedidos/admin'. The 'Authorization' tab is active, showing 'Bearer Token' as the auth type. The response status is '200 OK' with a response time of 1.75 s and a size of 770 B. The response body is displayed in JSON format, showing an array of order objects.

```
{
  "id": 11,
  "totalPrice": "0.03",
  "items": [
    {
      "id": 1,
      "name": "Teste",
      "price": 0.01,
      "quantity": 3
    }
  ],
  "status": "Pendente",
  "deliveryMethod": "retirada",
  "deliveryAddress": null,
  "createdAt": "2026-06-08T02:52:20.683Z",
  "updatedAt": "2026-06-08T02:52:20.683Z",
  "userId": 5
}
```

CRUD orders: /api/pedidos

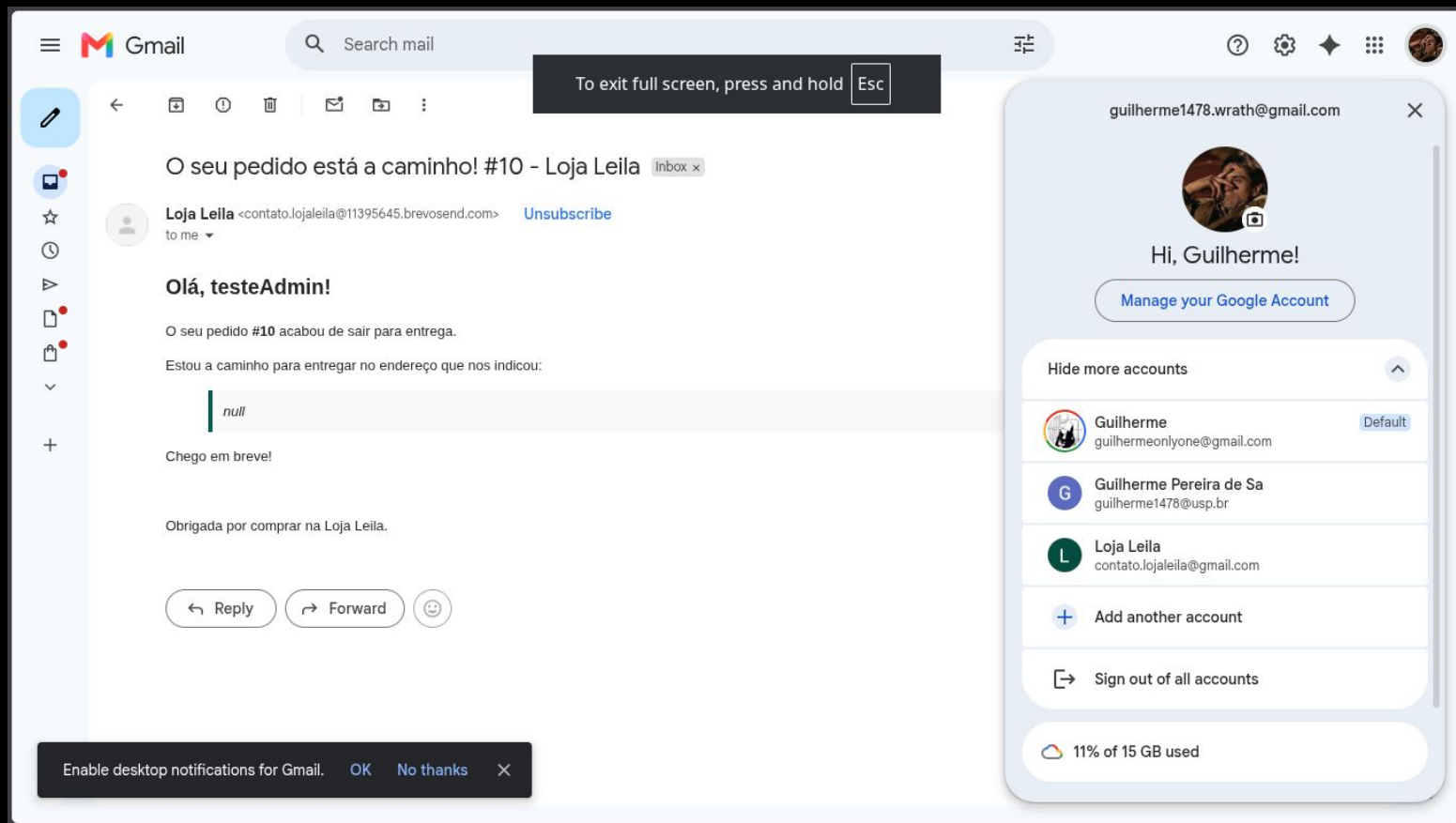
The screenshot shows a REST client interface with a sidebar on the left containing a tree view of collections: COLLECTIONS (Cart, CLOTHINGS, ORDERS), ORDERS (Get data, Post data, testando pedido confirmado, meus, New Request), and USERS. The main panel displays a PUT request to `https://api-lojalella.onrender.com/api/pedidos/10/status` with a raw JSON body:

```
{
  "status": "Saiu para Entrega"
}
```

. The response is a 200 OK status with a response time of 2.15 s and 782 B. The response body is shown in JSON format:

```
{
  "message": "Status atualizado para 'Saiu para Entrega' com sucesso! A cliente ser\u00e1 notificada.",
  "order": {
    "id": 10,
    "totalPrice": "0.02",
    "items": [
      {
        "id": 1,
        "name": "Teste",
        "price": 0.01,
        "quantity": 2
      }
    ],
    "status": "Saiu para Entrega",
    "deliveryMethod": "retirada",
    "deliveryAddress": null
  }
}
```

CRUD orders: /api/pedidos



CRUD orders: /api/pedidos

The screenshot displays the NeonDB web interface. At the top, the user 'Guilherme' is logged in with a 'Free' plan, and the database 'loja-leila-db' is selected. The interface includes a sidebar with navigation options: PROJECT (Dashboard, Branches, Integrations, Settings), BRANCH (production), and APP BACKEND (Data API, Auth). The main area is titled 'Tables' and shows the 'production' database with a 'public' schema. The 'orders' table is selected, and its structure is displayed in a table format. The table has five columns: 'id serial', 'totalPrice numeric(10, 2)', 'Items json', and 'status varchar(255)'. There are five records in the table, all with a status of 'Pendente'. The interface also includes a '+ Add record' button and a 'Collapse menu' button at the bottom left.

PROJECT

- Dashboard
- Branches
- Integrations
- Settings

BRANCH

- production

APP BACKEND

- Data API
- Auth

Tables

production

neondb

Schema

public

Search...

Clothings

orders

users

+ Add record

	id serial	totalPrice numeric(10, 2)	Items json	status varchar(255)
☐	1	0.01	[{"id":1,"name":"Teste...	Pago
☐	2	0.01	[{"id":1,"name":"Teste...	Pendente
☐	3	0.01	[{"id":1,"name":"Teste...	Pendente
☐	4	0.01	[{"id":1,"name":"Teste...	Pendente
☐	5	0.01	[{"id":1,"name":"Teste...	Pendente

Collapse menu

CRUD **orders**: /api/pedidos

The screenshot displays the NeonDB web interface. The top navigation bar includes the NeonDB logo, the username 'Guilherme', a 'Free' tier indicator, the database name 'loja-leila-db', and buttons for 'All OK', 'Ask AI', and 'Upgrade'. The left sidebar contains a 'PROJECT' section with links to 'Dashboard', 'Branches', 'Integrations', and 'Settings', and a 'BRANCH' section with 'production' selected. Below these are links for 'Overview', 'Monitoring', 'SQL Editor', 'Tables' (highlighted), 'Backup & Restore', and 'Data Masking'. The 'APP BACKEND' section includes 'Data API' and 'Auth'. The main area is titled 'Tables' and shows the 'production' database selected. Under the 'public' schema, the 'orders' table is highlighted. The table structure is shown with columns: 'id serial', 'username varchar(255)', 'password varchar(255)', and 'isAdmin boolean'. A single record is displayed with the following values: '1' for id, 'testeAdmin' for username, '\$2b\$10\$9Zef4zqZJ150Jpx...' for password, and 'TRUE' for isAdmin. The interface also includes a '+ Add record' button and pagination controls showing '1 - 1 of 1'.

PROJECT

- Dashboard
- Branches
- Integrations
- Settings

BRANCH

- production

Overview

Monitoring

SQL Editor

Tables

Backup & Restore

Data Masking **BETA**

APP BACKEND

- Data API
- Auth

Collapse menu

Tables

production

neondb

Schema

public

Search...

Clothings

orders

users

	id serial	username varchar(255)	password varchar(255)	isAdmin boolean
	1	testeAdmin	\$2b\$10\$9Zef4zqZJ150Jpx...	TRUE

+ Add record

1 - 1 of 1

UPLOAD DE IMAGENS!!!



```
"Clothing": {  
  "id": 1,  
  "name": "Vestido Floral Leila Elegante",  
  "price": "1.01",  
  "size": "G",  
  "image": "https://res.cloudinary.com/dhrzmy2wv/image/upload/v1780778306/lojaroupa_produtos/fpqhoikro57alwm933yw.  
    png",  
  "stock": 9  
}
```

❑ Imagem acima de como fica. **GET** do `/cart`.

❑ Fazemos upload local para o server deles criando uma roupa. Próximo slide.

UPLOAD DE IMAGENS!!!

The screenshot displays the Postman application interface. On the left, the sidebar shows a collection named 'CLOTHINGS' with a sub-collection 'Post data' selected. The main workspace shows a POST request to the endpoint 'api-lojalella.onrender.com/api/produtos/criar'. The request body is set to 'form-data' and contains the following data:

Key	Type	Value	Description
name	Text	Teste	
price	Text	00.01	
size	Text	M	
image	File	unnamed.png	
stock	Text	3	

The response is shown in the bottom panel, indicating a '201 Created' status. The JSON response is as follows:

```
1 {
2   "message": "Roupa cadastrada com sucesso!",
3   "clothing": {
4     "id": 4,
5     "name": "Teste",
6     "price": "0.01",
7     "size": "M",
8     "stock": 3,
9     "image": "https://res.cloudinary.com/dhrzmy2wv/image/upload/v1780952072/lojaroupa_produtos/qvrhnn6iwqsjq24rfyrd.png"
10  }
11 }
```

The bottom status bar shows 'Connect Git', 'Console', 'Terminal', and '0 0 0' errors. The right sidebar includes 'Globals', 'Vault', 'Tools', and other utility icons.

SITE NO AR!!

▲ URL do SITE:

📄 <https://lojaleila.onrender.com/>

▲ Como diria aquele velho ditado de um velho sábio:

"Site no ar é igual avião: ninguém elogia quando tá no ar, mas se ele cair..."



E o frontend teve atualizações?

- ❑ Descubra a partir do próximo slide.



- ❑ **Vitrine OLX**

- FIM +
- DEMONSTRAÇÃO -



OMG
Placeholder Text



LOJA LEILA

